signal_detector

Classroom phone-presence detector — setup and operating manual

July 24, 2026

1 What it is

Two ESP32-S3 boards act as passive 2.4 GHz radio sensors. Each one listens for WiFi and Bluetooth activity, and every ∼2 s reports what it heard to a Python program (`collector.py`) running in Termux on your phone. The phone keeps a *baseline* of the devices normally present in the empty room, then shows a live table of any *extra* devices, using the two boards to place each one in a rough corner of the room. It also tracks how much each device transmits and flags short *episodes* of use — a phone picked up for a moment — buzzing you when one starts near a board, and leaving a "used Xs ago" marker so you can step out for a few minutes and review what happened when you return. A web page (served by the phone) shows the same list, lets you mark a device *watch* or *disregard*, and graphs any device's traffic over several time windows (5 min up to 1 h 40 min), with episodes shaded.

Read this before trusting it. A phone that is *off* or in *airplane mode* emits nothing and is invisible — which is exactly what you tell students to do, so a careful cheater defeats it for free. Modern phones rotate randomized MAC addresses, so the device *count* is inflated (one phone can appear as several rows). Location is corner-level, never a single desk. Treat this as a live map of *active radios* plus a deterrent — not as proof, and not as a substitute for collecting phones at the door.

2 What you need

- **2× ESP32-S3-DevKitC-1** boards (WROOM-1).
- 2 USB-C cables and 2 power sources (power banks or chargers) for the room.
- A laptop with the **Arduino IDE** (only to flash the boards, once).
- An **Android phone** with **Termux** and **Termux:API** (both from F-Droid — see the note below).
- A WiFi network the phone and both boards can all join.
- *Optional, but buy two — one per board:* an **AD8317** (or AD8318) log power detector module with an SMA connector, plus a **600 MHz–6 GHz wideband whip** antenna (**SMA male**, sold for SDR/5G or drones — *not* the cheaper 700–2700 MHz LTE type, which misses 5G; see §8), and three jumper wires. About US$20 total. This adds the cellular arm of §8; without it everything else works as described.

Install **Termux** and **Termux:API** from the *same* source (both from F-Droid). They must be signed by the same key or the notifications will silently not work. Do not mix F-Droid and Play Store builds.

3 Part A — Flash the two sensors (once)

1. In Arduino IDE: install **esp32 by Espressif** (Boards Manager) and **NimBLE-Arduino** (Library Manager, version 2.x).
2. Select board **ESP32S3 Dev Module**. Set **USB CDC On Boot: Enabled** (so Serial works over USB-C).
3. Edit `secrets.h` with your WiFi and your phone's IP address (you get that IP in Part B):

```
#define WIFI_SSID   "your-wifi"
#define WIFI_PASS   "your-password"
#define BOARD_ID    1              // 1 on the first board, 2 on the second
#define SERVER_HOST "192.168.1.50" // your phone's LAN IP
#define SERVER_PORT 8000
#define RF_ENABLED  0              // 1 only if an AD8317 is wired (see the RF
    section)
#define RF_ADC_PIN  4              // ADC1 pin (GPIO1..GPIO10) the detector's VOUT
    goes to
```

4. Flash the first board with `BOARD_ID 1`. Then change it to `BOARD_ID 2` and flash the second board.
5. Label the boards "1" and "2". Board 1 goes front-left, board 2 goes back-right (or edit `BOARD_LABELS` in `collector.py` to match where you actually place them).

The boards have no buttons and no commands — once powered, they just sniff and POST. All control is on the phone.

4 Part B — Set up the phone (Termux)

1. Install the packages and the Python web framework:

```
pkg install python termux-api
pip install flask
```

2. Find the phone's LAN IP (the `inet` on `wlan0`) and put it in `secrets.h` as `SERVER_HOST` before flashing the boards:

```
ifconfig
```

3. Copy `collector.py` onto the phone (git clone the repo, or paste the file).
4. Read the built-in help any time:

```
python collector.py --help
```

5 Part C — Running an exam

1. Power both boards, one in each corner.
2. On the phone, keep it plugged in, and in Termux:

```
termux-wake-lock      # keeps it running with the screen off
python collector.py
```

3. Watch the table start to populate (that confirms the boards are reaching the phone).
4. With the room still **empty**, press `b` then Enter. It captures the ambient devices for 10 minutes, then switches to live automatically.
5. Let the students in. The table now lists only the *extra* devices, with a corner and a type. When a device is actively *used* (a traffic episode) near a board, the phone buzzes. Glance at the zone and walk that way; the `USED` column and the graph keep the moment if you look later.

If Android keeps killing Termux during a long exam: keep the phone plugged in, run `termux-wake-lock`, and set Termux to *Unrestricted* under Android's battery settings.

6 Reading the live table

Column	Meaning
MARK	Blank, `W` (watch), or `D` (disregard). Set on the web page.
MAC	Device hardware address (often randomized, so it changes over time).
RSSI	Strongest signal seen, in dBm. Closer to 0 = nearer.
ZONE	`near front-left` / `near back-right` / `center`.
TRAFFIC	Frames and bytes heard from this MAC in the last ∼2 s (WiFi only). A relative, undersampled "h
USED	`NOW` during a traffic episode, else "used Xs ago" for a few minutes after one; `present` for a BLE de
TYPE	`WiFi` or `BLE`, plus `rand` (randomized MAC) or `apple`.

A device is placed in a corner only when *both* boards hear it: the board with the stronger signal wins; if they are within a few dB it is called `center`. Watched (`W`) devices sort to the top and show red; disregarded (`D`) devices sort to the bottom and show green.

7 Episodes — catching a brief moment of use

The point of the exam isn't a phone being *on*, it's a phone being *used*: picked up, a query sent, an answer read, put down — a burst of tens of seconds (a query, or a photo of the question uploaded). The collector detects this as an **episode**: over the last 30 s, a device's WiFi traffic must clear two gates at once —

- **volume**: total bytes above a floor (`EPISODE_MIN_BYTES`), and
- **average frame size** above `EPISODE_MIN_FRAME`.

The frame-size gate is what separates real use from idle. An idle-but-connected phone still emits a trickle of *tiny* keepalive/ACK frames; a real transfer sends *large* data frames. Requiring a large average frame size means a device merely holding its WiFi connection alive never trips, while a genuine query does. 30 s matches the length of a brief use.

Each episode is **logged with a timestamp**, so this survives you leaving the room: step out to the bathroom for two or three minutes, and when you come back the device still shows "used 90s ago" and its graph shows a shaded hump at that time. When an episode starts on a device close to a board (or one you marked `W`), the phone buzzes.

BLE presence. Watches and earbuds show up over Bluetooth and don't generate WiFi traffic — for them the signal is simpler: a non-baseline BLE device is a violation *just by being present*. Those rows show yellow, are labelled `present`, sort up, and buzz you. (Since students shouldn't have any such device, its mere presence is the alert.)

Calibrating the floor

Do this once, in the real room, with a test phone that is *not* in the baseline. Select it on the web page and read the **calib** line under the graph — it shows the current 30 s window sum and average frame size, next to the floor and frame thresholds, and says `EPISODE` or `below`.

1. **Idle:** leave the phone connected but unused for a few minutes. Note the highest window sum it reaches (keepalive chatter).
2. **Query:** do a real query (and a photo upload) a few times; note the window sum.
3. Set `EPISODE_MIN_BYTES` **between** the idle peak and the smallest query. The frame-size gate handles the rest.

Calibrate from a worst-case desk — far from both boards. A phone far from a sensor is heard weaker and its frames are caught less often, so the *same* query yields a smaller window sum than one done next to a board. Set the floor from the far position or a corner query can slip under it.

8 The cellular arm (optional AD8317)

Everything above is 2.4 GHz. A phone with WiFi and Bluetooth switched off but mobile data on is invisible to it, because it transmits on entirely different bands (in Brazil roughly 700, 850, 900, 1800, 2100 and 2600 MHz, plus 3.5 GHz for 5G). The ESP32's radio cannot tune there. An **AD8317** log power detector can: it is a passive broadband RF voltmeter, the same principle as the commercial "phone detector" wands. Listening is legal; only *jamming* (transmitting to block phones) is not, and nothing here transmits.

Which bands you actually cover

The detector chip itself spans 1 MHz–10 GHz, so coverage is decided entirely by the antenna. These are the *uplink* halves — the phone's own transmission, which is what has to be caught:

Band	Uplink
B28 (700 APT)	703–748 MHz
B5 (850)	824–849 MHz
B8 (900)	880–915 MHz
B3 (1800)	1710–1785 MHz
B2 (1900)	1850–1910 MHz
B1 (2100)	1920–1980 MHz
B7 / B38 (2600)	2500–2620 MHz
n78 (5G)	3300–3800 MHz

Everything except n78 falls inside 703–2620 MHz, so a 700–2700 MHz LTE whip covers all of 2G, 3G and 4G with no filters. It does *not* cover n78 — and 5G standalone on 3.5 GHz is live in the Brazilian capitals, so a phone on *5G puro* transmits there and nowhere else. A **600 MHz–6 GHz wideband whip** costs the same and closes that gap.

Sensitivity is not flat across such a span: the detector's intercept shifts a few dB with frequency and a wideband whip's gain ripples several more, perhaps 5–10 dB end to end. Detection is relative to the baseline so this is harmless, but do not set `RF_MARGIN_DB` razor-thin, and read the dBm figure as a label, accurate only near the band you calibrated at.

How far it reaches

Range is set by the phone's *power control*, not by the band. A phone with a weak tower signal shouts at +23 dBm; one with a strong signal drops toward 0 dBm. At 1.8 GHz in free space, against the AD8317's ~ -55 dBm floor:

Phone TX	At 10 m	At 3 m
+23 dBm (poor coverage)	≈ -35 dBm, caught	caught easily
0 dBm (strong coverage)	≈ -58 dBm, **missed**	≈ -47 dBm, caught

So the usable radius swings from a few metres to the whole room. Thick-walled classrooms usually have poor indoor coverage, which pushes phones toward full power and works in your favour — but place the antenna among the students, not at the back wall.

Acceptance test — trust the room, not the datasheet

Wideband antennas are sold in housings identical to narrowband ones, and sellers mix up the specifications, so measure the thing you actually received. After wiring it and running a baseline:

1. Take a test phone, switch *off* its WiFi and Bluetooth, leave mobile data on. Now the only thing it can transmit is cellular.

2. Stand beside the board and start a speed test or a large upload. The RF row should go `BURST` within a couple of seconds.
3. Walk backwards a metre at a time, repeating, until it stops firing.

That distance is your real detection radius — on the band your carrier actually assigned, through the walls and bodies of that room. A few metres is normal and usable. If it only fires within arm's reach, you were sold a narrowband element and should return it.

Wiring and enabling

1. Screw the whip onto the module's SMA connector.
2. Module `VOUT` → `RF_ADC_PIN` (default GPIO4), `3V3` → `3V3`, `GND` → `GND`. The output swings up to about 2 V, inside the ADC's range, so no divider.
3. Set `RF_ENABLED 1` in that board's `secrets.h` and reflash. On boot the Serial prints `RF arm on GPIO4`.
4. Run a *new* baseline (`b`) with the antenna in place.

SMA, not RP-SMA. Antennas sold for drones and FPV gear are usually *reverse polarity*: same threads, but a socket where the centre pin should be. An RP-SMA antenna screws onto the detector and carries no signal whatsoever — the RF row simply reads `quiet` forever, with nothing to suggest a fault. Before buying, confirm the antenna has a visible pin in the middle. Longer is better at the low end: take a 200 mm whip over a 178 mm one.

Fit both boards if you can. A single detector does not cover a classroom — it covers its own corner, because the reach against a low-power phone is only a few metres (see above). Two of them roughly double the watched area, and since the reach is short, a burst seen by one board and not the other already tells you which half of the room it came from. Each board keeps its own ambient level, its own burst decision and its own cooldown, and prints its own `RF` line; set `RF_ENABLED 1` in both. One module still works, it simply watches less. Keep the whip away from the ESP32's own antenna: the board blanks its sampling while it POSTs, but a whip taped against the WROOM module will still read mostly the ESP32.

How it reads

The board samples the detector at about 5 kHz — fast enough that a 577 μs GSM slot or a 1 ms LTE subframe spans several samples — and every 2 s reports the interval's loudest sample, its quiet level, and how many samples sat clearly below the quiet level (the detector is *inverting*: more RF power means *lower* voltage). The collector converts to dBm and compares against the ambient level learned during the baseline. When a batch runs `RF_MARGIN_DB` above ambient with at least `RF_MIN_HITS` burst samples, it is a **burst**: the console and web page show `BURST +x.x dB` and you get one buzz (then a cooldown). Between bursts the row reads `quiet`, or "last burst 40s ago" for a while afterwards.

If `RF_MARGIN_DB` at 3 dB fires on nothing, raise it; if the room's own WiFi keeps setting it off, raise it and re-baseline with the classroom in its normal state. Absolute dBm values depend on `RF_SLOPE_MV_DB` and `RF_V_REF_MV` (nominal AD8317 figures) — they are labels only; the detection is relative to the baseline and does not depend on them being right.

What it does and doesn't buy you

It catches a phone *actively transmitting* on any band in the table above: sending a message, loading a page, uploading a photo of the question. That is the cheating behaviour, and it is caught even with WiFi and Bluetooth off. But without a band filter it is deliberately unselective — it reports any strong transmitter — it names no device and no desk, its reach shrinks to a few metres against a phone with strong tower coverage, an idle phone still emits almost nothing between tracking updates, and airplane mode remains invisible. Read it as a second, independent yes/no next to the WiFi table: a burst the device list cannot explain is very likely a phone on mobile data.

9 Web dashboard and traffic graphs

The collector serves a small web page. On the phone open `http://localhost:8000/`; from your laptop on the same WiFi open `http://<phone-ip>:8000/`. Nothing is installed and no internet is needed. It gives you three things the console can't:

- **Marking.** Each row has `W` and `D` buttons. Mark a device `W` (watch) and it turns red and sorts to the top; mark it `D` (disregard — e.g. your own phone, or one you already checked) and it turns green and sorts to the bottom. Unmarked devices sit in the middle. Marks persist across restarts (`marks.json`).
- **Traffic graphs with time windows.** Click a MAC to graph its traffic. Buttons pick the window: `5m`, `15m`, `30m`, `60m`, `1h40m`. The short windows are for reading a single episode; the long one is the whole picture.
- **Episodes shaded.** Detected episodes appear as shaded humps on the graph, so you can see exactly when a device was active.
- **Calib readout.** Below the graph, a `calib` line shows the selected device's current 30 s window sum and average frame size against the thresholds — used to calibrate the floor (see above).

The long graph is only meaningful for *stable* MACs (no `rand` tag) — a laptop, or a phone associated to an access point. A randomized MAC is discarded and replaced by the phone every few minutes, so its history only ever spans those few minutes. You are watching a short-lived identity, not a device across the whole exam.

10 collector.py — command reference

Type the letter in the Termux console, then press Enter.

Key	Action
`b`	Start baseline capture (10 min). Run in the **empty** room.
`l`	Live detection.
`i`	Idle (keep receiving, stop deciding).
`s`	Save baseline to `baseline.json`.
`c`	Clear baseline.
`+`	Raise RSSI threshold by 5 dBm (fewer, closer devices).
`-`	Lower RSSI threshold by 5 dBm (more, farther devices).
`q`	Quit.

Tunables (top of `collector.py`)

Constant	Meaning
`RSSI_THRESHOLD`	Ignore devices weaker than this (default −80 dBm).
`ALERT_STRONGEST_MIN`	Only buzz on episodes at least this close (−70 dBm), unless `W`.
`ALERT_COOLDOWN_SECS`	Minimum gap between buzzes, per MAC (default 60).
`BASELINE_SECS`	Baseline capture length (default 600 = 10 min).
`EXPIRE_SECS`	Forget a device unseen this long (default 120).
`BOARD_LABELS`	Which corner each board sits in.
`HISTORY_SECS`	How much traffic history is kept (default 6000 = 1 h 40 min).
`EPISODE_WINDOW_SECS`	Detection window (default 30 s; a brief phone use).
`EPISODE_MIN_BYTES`	Window traffic must clear this floor (bytes/30 s). Calibrate.
`EPISODE_MIN_FRAME`	. . . and average frame size must clear this (default 250 B).
`EPISODE_RETAIN_SECS`	Keep "used Xs ago" this long after an episode (default 300).
`RF_MARGIN_DB`	RF burst = this far above the empty-room ambient (3 dB). Calibrate.
`RF_MIN_HITS`	. . . and at least this many burst samples per batch (default 3).
`RF_ALERT_COOLDOWN`	Minimum gap between RF buzzes, per board (default 60).
`RF_SLOPE_MV_DB`	Detector slope, mV/dB (22 for AD8317, 25 for AD8318).
`RF_V_REF_MV`	Detector output at 0 dBm. Affects the dBm label only.

11 Troubleshooting

Symptom	Fix
Table stays empty	Boards can't reach the phone. Check `SERVER_HOST` matches the phone's current IP, both boards and the phone are on the same WiFi, and the phone isn't on mobile data. On the board's Serial you should see `[POST] ... HTTP 200`.
No notification buzz	Install the **Termux:API** app (not just the `termux-api` package) from F-Droid, same source as Termux.
Termux dies mid-exam	Plug in the phone, run `termux-wake-lock`, set Termux battery to Unrestricted.
Huge device count	Randomized MACs. Raise the threshold with `+`, and read the count as "activity", not "number of phones".
Everything shows in one corner	The boards may be too close together or one is dead. Separate them; check both show `HTTP 200`.
RF row stays `quiet` even with a phone on a call beside it	First suspect the connector: an **RP-SMA** antenna mates mechanically and passes nothing (look for a centre pin). Otherwise check the Serial printed `RF arm on GPIO`*n*, that `VOUT` goes to an **ADC1** pin (GPIO1–10), and that the baseline was run *with* the antenna attached.
No RF row at all	That board has `RF_ENABLED 0`, or it is not reporting — the row only appears while batches arrive.

12 Honest limits

- **Off / airplane mode = invisible.** No radio, no detection.
- **Randomized MACs inflate the count.** One phone → several rows.

- **Zoning is coarse.** A corner, not a desk; bodies and reflections blur the signal.
- **2.4 GHz only**, unless the AD8317 arm is fitted — and that one sees *any* transmitter, never a named device, and only within a few metres when the phone is transmitting at low power.
- **An idle phone is nearly silent** on every band. Both arms catch *use*, not mere presence.

It is a proctoring aid and a deterrent. The only method that actually removes phones is collecting them physically at the door.